



14.6. Quick Sort

Previous Section:

 [14.5. Merge Sort](#)

Contents:

[1. The Quick Sort Algorithm](#)

[2. Demonstration of Quick Sort](#)

[3. Implementation of Quick Sort](#)

[Summary](#)

[References](#)

Quick Sort is a **Divide-and-Conquer** sorting algorithm that selects one element, called the **pivot**, and rearranges the array so that:

- All elements **smaller than the pivot** are placed to its left.
- All elements **greater than the pivot** are placed to its right.

After this rearrangement (called **partitioning**), the pivot is already in its correct sorted position. The algorithm then recursively applies the same process to the left and right sub-arrays, excluding the pivot itself.

The pivot can be chosen in several ways, such as:

- The first element
- The last element
- The middle element
- A random element
- The median (median-of-three)

The efficiency of Quick Sort depends greatly on how well the pivot divides the array into balanced sub-arrays.

In the average case, Quick Sort runs in $O(n \log n)$ time, making it one of the fastest sorting algorithms in practice.

However, if poor pivots are consistently chosen (for example, always selecting the smallest or largest element), the algorithm degrades to $O(n^2)$ time.

Since the partitioning is performed in-place, Quick Sort requires only $O(1)$ auxiliary memory (excluding the recursive call stack).

1. The Quick Sort Algorithm

The most important step of Quick Sort is **partitioning**, which places the pivot into its correct position while separating smaller and larger elements. There are two common partitioning methods.

- **Method 1 – Two pointers from opposite ends (Hoare Partition):** Choose a pivot, then use two pointers.
 - Pointer **i** moves from **left to right**, searching for an element **greater than the pivot**.
 - Pointer **j** moves from **right to left**, searching for an element **less than or equal to the pivot**.

Whenever **$i < j$** , the two elements are on the wrong sides of the pivot, so they are swapped.

When the two pointers meet or cross, the partition is complete. The algorithm then recursively sorts the left and right partitions.

- **Method 2 – Two pointers moving together (Lomuto Partition):** Choose a pivot (commonly the last element).
 - Pointer **i** keeps track of the boundary between elements smaller than the pivot and the remaining elements.
 - Pointer **j** scans the array from left to right.

Whenever **$a[j] < \text{pivot}$** , increment **i** and swap **$a[i]$** with **$a[j]$** , then move both **i** and **j** to the next element. Else, increment **j** and compare again.

After scanning the array, place the pivot immediately after the last smaller element.

This approach is simpler to implement and is commonly used in textbooks.

- **Time Complexity**

- Best Case: $O(n \log n)$
- Average Case: $O(n \log n)$
- Worst Case: $O(n^2)$

- **Space Complexity**

- $O(1)$ auxiliary space (excluding recursion)

2. Demonstration of Quick Sort

Suppose we want to sort the following array using the **Lomuto Partition** method, where the **last element is chosen as the pivot**.

- Initial Array

Pivot = 5

8	2	4	7	1	3	9	6	5
---	---	---	---	---	---	---	---	---

↑i ↑j

- Step 1: Compare **8** with the pivot **5**. Since **8 > 5**, do nothing. Move **j** to the next element.

8	2	4	7	1	3	9	6	5
---	---	---	---	---	---	---	---	---

↑i ↑j

- Step 2: Compare **2** with the pivot. Since $2 < 5$, increment **i** swap **a[i]** and **a[j]**. Move **j** to the next element.

2	8	4	7	1	3	9	6	5
---	---	---	---	---	---	---	---	---

↑i ↑j

2	8	4	7	1	3	9	6	5
---	---	---	---	---	---	---	---	---

↑i ↑j

- Step 3: Compare **4** with the pivot. Since $4 < 5$, increment **i** and swap it into the left partition. Move **j** to the next element.

2	4	8	7	1	3	9	6	5
---	---	---	---	---	---	---	---	---

↑i ↑j

2	4	8	7	1	3	9	6	5
---	---	---	---	---	---	---	---	---

↑i ↑j

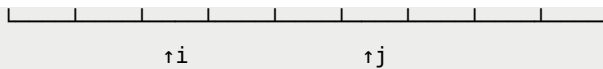
- Step 4: Compare **7**. Since $7 > 5$, no swap is needed. Move **j** to the next element.

2	4	8	7	1	3	9	6	5
---	---	---	---	---	---	---	---	---

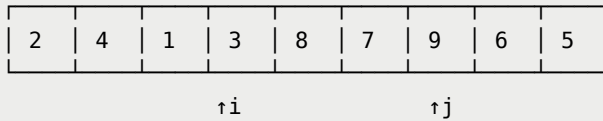
↑i ↑j

- Step 5: Compare **1**. Since $1 < 5$, increment **i** swap it with the first element greater than the pivot. Move **j** to the next element.

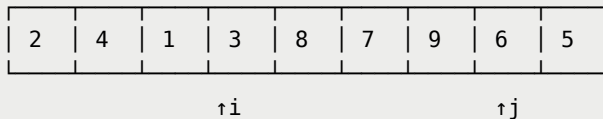
2	4	1	7	8	3	9	6	5
---	---	---	---	---	---	---	---	---



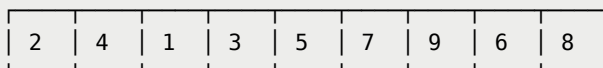
- Step 6: Compare **3**. Since **3 < 5**, increment **i** and swap again. Move **j** to the next element.



- Step 7: Compare **9**. Since **9 > 5**, do nothing. Move **j** to the next element.

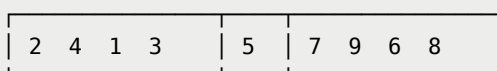


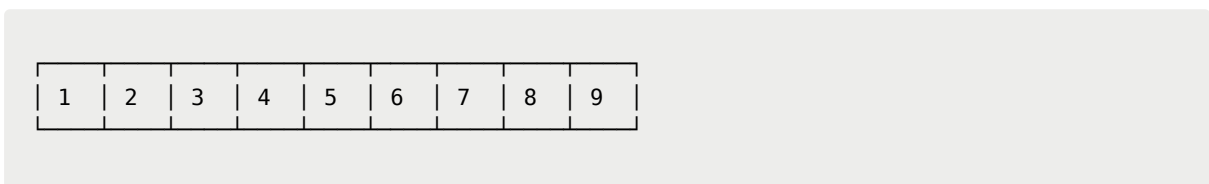
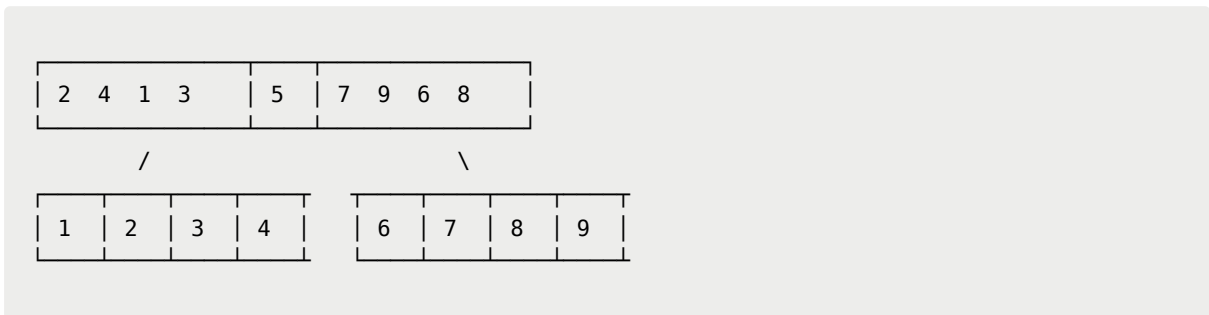
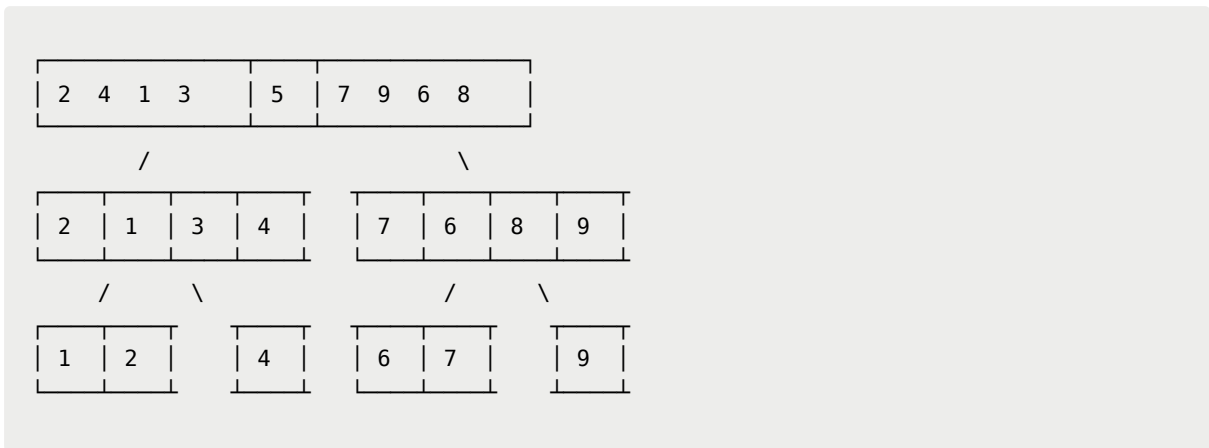
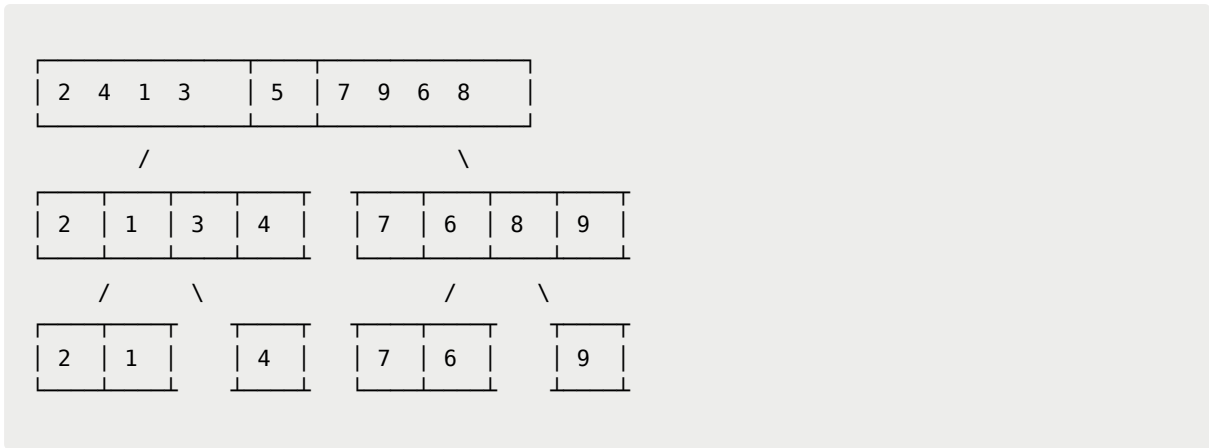
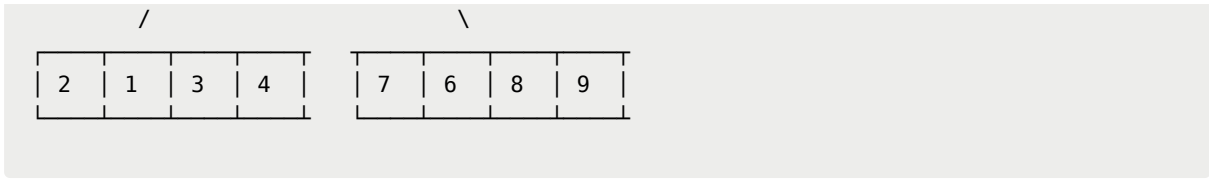
- Step 8: Compare **6**. Since **6 > 5**, do nothing. Pointer j now points to the pivot. The scan is complete.
- Final Pivot Placement: Swap the pivot with the first element greater than the pivot.



The pivot **5** is now in its final sorted position. The array has been divided into two independent sub-arrays.

Quick Sort now recursively sorts the left and right sub-arrays using the same partitioning process until every sub-array contains at most one element.





3. Implementation of Quick Sort

Quick Sort is naturally implemented using **recursion**.

```
def partition(a): # Partition algorithm
    low = 0
    high = len(a) - 1
    pivot = a[high]
    i = low - 1 # Indicate the right position of pivot found so far
    for j in range(low, high):
        if a[j] < pivot: # If the current value at index j is lower than pivot
            i += 1 # The "i" is incremented by 1
            a[i], a[j] = a[j], a[i] # Swap index i and index j
    a[i + 1], a[high] = a[high], a[i + 1] # Swap index i + 1 to the pivot
    return i + 1 # Return the correct position of the pivot

def quick_sort(a): # [Time(n), Space(n) = O(n ^ 2), O(1)]
    low = 0
    high = len(a) - 1
    if low < high:
        pos = partition(a)
        # Divide the array into two parts, excluding the pivot
        # Recursively call the function will trigger partition again
        quick_sort(a[low:pos])
        quick_sort(a[pos+1:high+1])
    return a
```

Each recursive call performs the following steps:

1. Select a pivot.
2. Partition the array so that smaller elements are placed to the left of the pivot and larger elements to the right.

3. Recursively sort the left partition.
4. Recursively sort the right partition.

The recursion stops when a partition contains **zero or one element**, since such a partition is already sorted.

Depending on the implementation, Quick Sort commonly uses either the **Hoare Partition** or the **Lomuto Partition** scheme. Both achieve the same objective but differ in how the pointers move and how elements are swapped.

Summary

Quick Sort is a Divide-and-Conquer sorting algorithm that repeatedly selects a pivot, partitions the array around that pivot, and recursively sorts the resulting sub-arrays.

On average, it achieves an excellent time complexity of $O(n \log n)$ while requiring only $O(1)$ auxiliary memory for partitioning, making it one of the fastest in-place sorting algorithms in practice.

However, its performance depends heavily on pivot selection, as poor pivots can result in a worst-case time complexity of $O(n^2)$.

Despite this limitation, Quick Sort remains one of the most widely used sorting algorithms due to its simplicity, efficiency, and excellent average-case performance.

References

https://drive.google.com/file/d/1jK22rfWszKKgPgeprquGoeJKqgBg8rUY/view?usp=drive_link

<https://www.geeksforgeeks.org/dsa/quick-sort-algorithm/>

<https://www.programiz.com/dsa/quick-sort>

<https://www.youtube.com/watch?v=7h1s2SojIRw&t=21s>

<https://www.youtube.com/watch?v=Vtckgz38QHs&t=183s>

Next Section:

 [14.7. Shell Sort](#)

